

Индивидуальные практические задания по использованию команды `find`

Ниже приведены **задания**, каждое из которых требует цепочку из нескольких ключей `find` (фильтры по типу, имени, размеру, времени, глубине, шаблонам) и, при необходимости, обработку через `-exec` / `-ok`.

- В `/var/log` найти файлы `*.log` размером больше 1 МБ, изменённые за последние 7 дней, и подсчитать строки в каждом (`wc`).
- В домашнем каталоге найти скрытые каталоги глубиной не выше 2 и вывести их пути.
- В `/usr/bin` найти обычные файлы, изменённые более 180 дней назад, и вывести имя и время изменения.
- В `/home` найти файлы `*.py`, исключая каталоги `.venv` и `__pycache__`, и вывести путь.
- В `/tmp` найти пустые файлы и пустые каталоги и удалить их с подтверждением.
- В `/var` найти файлы размером больше 100 МБ и вывести их размер в байтах и путь.
- В `/etc` найти файлы `*.conf`, изменённые за последние 24 часа, и вывести время изменения.
- В `/usr/lib` вывести список каталогов глубиной до 2 уровней.
- В `/var` найти файлы больше 10 МБ, изменённые ровно 2–3 дня назад, и вывести размер в байтах.
- В `/opt` найти каталоги с именем `cache` на глубине 2–4 и удалить их содержимое, не удаляя сами каталоги.
- В дереве проекта исключить `build` и `vendor`, найти файлы без расширения и вывести размер и путь.
- В `/usr/share` вывести каталоги с именем `locale` не глубже 3 уровней.
- В `/` найти блочные и символьные устройства и вывести путь и тип.
- В `/var/www` вывести файлы, изменённые раньше, чем `baseline.txt`.
- В `/tmp` найти обычные файлы старше 2 дней и обнулить их (`truncate`).
- В `/home` вывести каталоги, чьё имя начинается с цифры, глубина 1–2.
- В проекте найти файлы с расширением `.json` размером от 1 до 50 КБ, изменённые не более недели назад.
- В `/etc` вывести каталоги, созданные за последние 24 часа.
- В `/home` найти файлы с кириллицей в имени и вывести пути.
- В проекте найти все файлы `LICENSE`, исключая `.git` и `node_modules`.
- В `/home` найти `*.bak` старше 30 дней и удалить с подтверждением.

22. В `/etc` вывести файлы старше 180 дней с датой изменения.
23. В `/home` найти каталоги с именем `cache` на любой глубине и удалить все файлы с расширением `.tmp` внутри них.
24. В `/home` найти `*.csv` и вывести первые 3 строки каждого (`head`).
25. В `/home` вывести каталоги глубиной до 2, имя которых содержит `backup`.
26. В проекте исключить `tests` и найти `.py` больше 50 КБ.
27. В `/var/log` найти `*.log` с временем доступа больше 7 дней и временем изменения меньше 2 дней и вывести путь.
28. В `/home` найти пустые файлы и удалить их.
29. В `/var` вывести файлы, обновлённые новее заданной даты.
30. В проекте вывести файлы, имя которых состоит только из заглавных букв, цифр и `_`.
31. В `/home` вывести `*.json`, новее контрольного файла `check.json`.
32. В `/var/tmp` найти файлы, имена которых соответствуют шаблону `tmp-?????.dat`, и не старше 48 часов.
33. В проекте исключить каталоги `node_modules` и `dist`, найти файлы с расширением `.png` размером от 100 до 500 КБ.
34. В `/srv` найти каталоги глубиной 3–5, оканчивающиеся на `data`, и вывести их пути.
35. В `/opt` найти файлы, у которых время доступа новее времени изменения.
-

Критерии оценки

1. Логичная цепочка исполнения `find` – 40%

- Логичная комбинация ключей: тип (`-type`), имя (`-name`/`-regex`), глубина (`-mindepth`/`-maxdepth`), размер (`-size`), время (`-mtime`/`-ctime`/`-atime`), ограничения (`-prune`), формат (`-printf`), и т.п.

2. Корректность результата – 30%

- Команда реально находит/обрабатывает именно то, что описано.

3. Точность и безопасность действий – 20%

- Аккуратное использование `-exec`/`-ok`/`-delete`; отсутствие нежелательных побочных эффектов; тестирование через `-print` перед удалением/изменением.

4. Стиль и читаемость – 10%

- Скобки, `-o`/`-a`, экранирование, порядок фильтров, понятный `-printf`.